

Contents

1	Introduction	2
1.1	Problem to solve	2
2	Jetson and Local system connection	2
3	Docker	3
3.1	Some Helpful Docker Commands	3
3.1.1	Copying file from local to container	3
3.1.2	Docker Commit	4
4	First Test	4
5	Opti-Track system	4
5.1	Ros Package for opti track	5
6	Collecting Data through rosbags	7
6.1	Basic commands	7

1 Introduction

The link to the entire documentation of the car is indicated here. <https://roboracer.ai/build>.

The Jetson on board is a **Jetson Xavier NX**. We attach a separate Wifi Dongle but it still doesn't have Bluetooth. The Jetson runs an **Ubuntu 20.04** platform with **ROS2 Foxy**

Note: If your laptop/pc (called Local system from now) does not have Ubuntu 20.04 installed and does not run ros2 foxy use docker containers to do the same.

1.1 Problem to solve

The joystick cannot be connected to the jetson directly visa Bluetooth. Hence one way is to attach it to the local system and then forward the commands to the jetson.

In a broad sense, the joystick will be connected to the local system running the docker container. The joy node will begin in the docker container on the local system. Ros on the docker container and the jetson should have the same domain id.

Set **ROS_DOMAIN_ID = 10** in both the jetson and local system (write it in the bashrc file). Having same ros domain id's and running the same versions of ros can help directly communicate with two systems.

We would also have to run the docker container with special privileges so that it can access the hardware (joy stick)

2 Jetson and Local system connection

There are multiple ways to do this. The way shown in this documentation is different from the one shown in the fltenth's documentation.

In this scenario we chose to make the local system a hotspot and make jetson connect to it with a static IP every time it boots up. This is because later on we need to access the opti-track system. To do so we must be on the same network as the opti-track. Connection to a common WiFi hence will not work thus the best way is to make your local system a hotspot while connecting the the opti-track system network via Ethernet.

To set it up connect the jetson to a monitor along with a keyboard and mouse and connect to the local system hotspot manually.

Note: if it keeps asking password to be entered even tho the password is correct check the security type of the hotspot (by default is WPA3 but the hotspot is on WPA). Then open up a terminal and do the following

```
1 nmcli connection modify "MyJetsonHotspot" \  
2   wifi-sec.key-mgmt wpa-psk  
3   ipv4.method manual \  

```

```

4  ipv4.addresses "10.42.0.100/24" \
5  ipv4.gateway "10.42.0.1" \
6  ipv4.dns "8.8.8.8" \
7  connection.autoconnect yes

```

ipv4 gateway being 10.42.0.1 is common for Ubuntu system (the Local system here was running ubuntu). And the static IP address set for the jetson is 10.42.0.100.

The above terminal command will make sure that u connect to "MyJetsonHotspot" every time on boot.

3 Docker

To set up a docker container create a Dockerfile and build it.

```

1  # Start from the official ROS 2 Foxy base image
2  FROM ros:foxy
3
4  # Install nano, joystick, and ros-foxy-joy package
5  RUN apt-get update && apt-get install -y \
6      nano \
7      joystick \
8      ros-foxy-joy \
9      && rm -rf /var/lib/apt/lists/*
10
11 # Set ROS environment and domain ID in a new bashrc (or overwrite
12 )
13 RUN echo "source_/opt/ros/foxy/setup.bash" >> /root/.bashrc && \
14     echo "export_ROS_DOMAIN_ID=10" >> /root/.bashrc
15
16 # Set default shell
17 CMD ["/bin/bash"]

```

Ideally u want one docker file to have all the information you have and build it but in this case due to a lot of experimentation we will just do further changes inside the docker container and then commit it.

To run this container with privileges use(make sure sudo not needed for docker) :

```

1  docker run -it --rm --net=host --privileged -v /dev/input:/dev/
    input foxy_custom

```

3.1 Some Helpful Docker Commands

3.1.1 Copying file from local to container

Incase you have a file in your local system and u want to copy it into the docker container then:

```

1  docker cp /home/shriram/Documents/Aalto_Intern/NatNet_folder/
    python_natnet/scripts/mocap.py focused_babbage:/opti-track_ws/
    src/mocap_publisher/mocap_publisher

```

Similarly to copy a folder from docker container to local system use:

```
1 docker cp inspiring_raman:/loop /home/shriram/Documents/  
Aalto_Intern/rosbags
```

Here:

- /home/shriram/Documents/Aalto_Intern/NatNet_folder/python_natnet/scripts/mocap.py is the path to the file **mocap.py** on the local system which is to be copied
- /opti-track_ws/src/mocap_publisher/mocap_publisher is the folder to file should be copied into.
- **focused_babbage** was the container name at that instance.

3.1.2 Docker Commit

```
1 docker commit abd1c3cd8c82 foxy_custom_2:updated_mocap
```

We can either commit it to a completely different image name in that case instead of foxy_custom_2 we can use another name, or we can use the same name with a different tag like the updated_mocap tag used here.

abd1c3cd8c82 is the runnign container id. And always make sure to update the alias in the bashrc file if you are using one.

4 First Test

Just open few terminals and login to the docker container in one terminal. To enter another session of the same container use

```
1 docker exec -it container_name bash
```

Make sure that the joystick was connected to the local system before running the docker container. Now run the joy node.

In another terminal use the following command and enter the password:

```
1 ssh -X fltenth@10.42.0.100
```

the fltenth stack using the bringup launch file. In this case **bringup_launch.3.py** because the lidar was different and we are also not running the joy node on the jetson.

Press the deadmans button which is the left top button on the behind and then using the joystick to control the vehicle. If everything was done properly this should work.

5 Opti-Track system

The opti track system is a **NaturalPoint Inc** opti track system. The software it runs is the **motive software v2.0.0**. The configurations are given below:

```

1 connection_type: "Multicast"
2 server_address: "130.233.123.181"
3 local_address: "130.233.123.221" (local system address)
4 multicast_address: "239.255.42.99"
5 server_command_port: 1510
6 server_data_port: 9000

```

We need to be able to feed the data streamed from the motive software right into the jetson so that it can use the state estimates given by the opti track sensor.

For this we use a python script converted into a ros package. The reason we do is is that ros2 foxy doesn't have any stable ros packages built for receiving the data transmitted by motive.

This method can be used to create a ros package for any distro. This is a general way to get information from motive. We use a python package called **NatNet**. The git-hub link is [NAT-NET](#) and the documentation page is [DOCUMENTATION](#).

Create a folder for downloading this package on the docker container. And then inside the terminal do the following:

```

1 git clone https://github.com/mje-nz/python_natnet
2 python setup.py install #inside the folder after the clone

```

5.1 Ros Package for opti track

Your natnet package is set up. Now make a ros package to get the data. The only python script you need to put in the ros package is the following

```

1 #!/usr/bin/env python3
2
3 import threading
4 import rclpy
5 import natnet
6 from rclpy.node import Node
7 from geometry_msgs.msg import PoseStamped
8 from std_msgs.msg import Header
9
10 TARGET_ID = 46 #Change as per ID
11
12 class MocapClientNode(Node):
13     def __init__(self, server_ip):
14         super().__init__('mocap_client_node')
15         self._latest_pose = None
16
17         self._pose_pub = self.create_publisher(PoseStamped, '
18             mocap/rigid_body', 10)
19
20         self._client = natnet.Client.connect(server_ip)

```

```

21     if self._client is None:
22         raise RuntimeError("Failed to connect to NatNet
23                               server")
24
25     self._client.set_callback(self.callback)
26
27     # Start NatNet client spin in a background thread
28     self._natnet_thread = threading.Thread(target=self.
29         _client.spin, daemon=True)
30     self._natnet_thread.start()
31
32     # ROS timer at 100 Hz to publish latest pose
33     self._timer = self.create_timer(1.0 / 100.0, self.
34         publish_latest_pose)
35
36 def callback(self, rigid_bodies, markers, timing):
37     for b in rigid_bodies:
38         if b.id_ != TARGET_ID:
39             continue
40
41         msg = PoseStamped()
42         msg.header = Header()
43         msg.header.stamp = self.get_clock().now().to_msg()
44         msg.header.frame_id = "mocap"
45
46         msg.pose.position.x, msg.pose.position.y, msg.pose.
47             position.z = b.position
48         msg.pose.orientation.x, msg.pose.orientation.y, msg.
49             pose.orientation.z, msg.pose.orientation.w = b.
50             orientation
51
52         self._latest_pose = msg
53         break
54
55 def publish_latest_pose(self):
56     if self._latest_pose is not None:
57         self._pose_pub.publish(self._latest_pose)
58         # Optional: Uncomment for debug
59         #self.get_logger().info("Published pose")
60
61 def main():
62     rclpy.init()
63     SERVER_IP = "130.233.123.181" # Mocap server IP
64
65     try:
66         node = MocapClientNode(SERVER_IP)
67         rclpy.spin(node)
68     except natnet.DiscoveryError as e:
69         print(f"Discovery error: {e}")
70     except Exception as e:
71         print(f"Unexpected error: {e}")

```

```

66     finally:
67         node.destroy_node()
68         rclpy.shutdown()
69
70 if __name__ == '__main__':
71     main()

```

NOTE: Do not forget to make this file executable, add a shebang at the start of the script and also change the setup.py file.

Make sure to commit the file as a new docker image and change the alias in the start to run this image.

NOTE: This ros package was made such that the fltenth car would be of the id = 46 and will be a rigid body. Further changes can be done by looking into the example python file in the Documentation of the natnet package.

6 Collecting Data through rosbags

6.1 Basic commands

```

1  ros2 bag record -a -o <bag_file_name>

```

-a tells to record all the available topics and **-o** lets you pick the name and location.

```

1  scp -r fltenth@10.42.0.100:/home/fltenth/rosbag_name /home/
    shriram/Documents/Aalto_Intern/rosbags

```